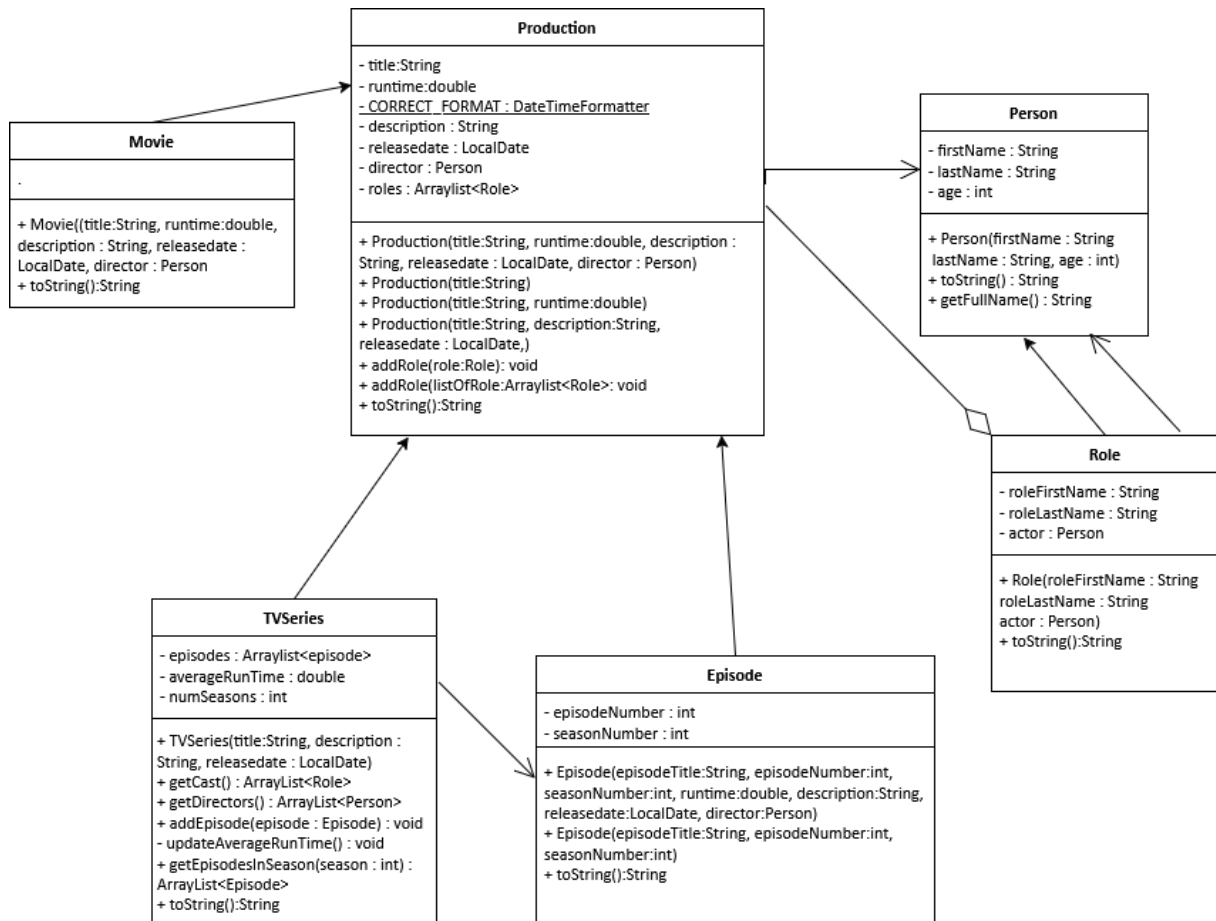




Oppgave 2.1:

<https://gemini.google.com/share/5275c4617950>

brukte KI og W3Schools som hjelpemidler

oppgave 2.2, 2.3 og 2.4:

<https://gemini.google.com/share/060c921e30b0>

brukte KI, W3Schools og reddit som hjelpemidler

oppgave 3.4:

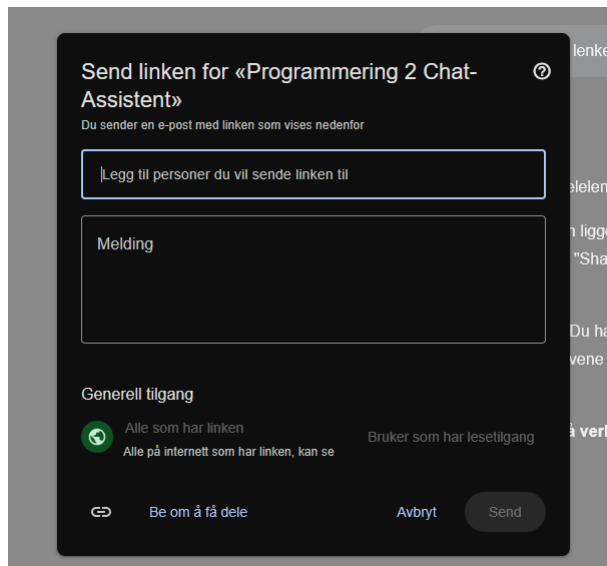
versjon 1:

<https://gemini.google.com/share/e7f096583ade>

brukte umodifisert KI og Oracle sin dokumentasjon som hjelpemidler

klarte halvveis, men ga opp ettersom det var kveld og hjernecellene sa «nah fam, sleeptime».

Dessverre fikk jeg ikke til å genere en lenke ved å trykke på dele-knappen, dette er hva som vises istedenfor:



Så her er samtalen for versjon 2 av 3.4 og oppgave 2.5, 2.6, 2.7 og 3.1:

Du sa

"Oppgave 2.5 - Roller

Nå ønsker vi også en oversikt over roller (og de som spiller dem). Lag derfor en klasse Role. Denne bør minst inneholde: roleFirstName, roleLastName og actor (av typen Person, hvem som spiller rollen).

Eksempel- Tony Stark, Robert Downey. Jr

Det er relevant å anta at en Production (Episode eller Movie), kan inneholde mange roller. Lag derfor en instansvariabel kalt roles som inneholder en liste med roller i Production. Lag en relatert get-metode (bør returnere en kopi av listen). Lag deretter to metoder ved bruk av overloading:

addToRoles(Role role) - Legger til én rolle

addToRoles(ArrayList<Role> listOfRoles) - Legger til flere roller på en gang

Test dette i Main.java ved å lage noen relaterte roller og legg

de til i en film ved hjelp av disse metodene. Hent så ut alle rollene

til filmen og skriv de ut.

Oppgave 2.6 - toString()

@Override toString() i alle klasser som ikke har det allerede og tilpass denne for den gitte klassen/barneklassen.

Oppgave 2.7 - Hente alle roller

Vi ønsker å vite om alle roller (og da også skuespillere) som har spilt i en TV-serie. Lag en metode i TVSeries

som heter `getCast()`, som returnerer en liste med roller. Du må altså

hente alle roller fra alle episoder som tilhører TV-serien.

Skriv så ut navnet på rollene, og skuespillerne som spiller disse.

(Du vil mest sannsynlig få samme skuespiller og rolle flere ganger, siden de spiller i flere episoder, dette er OK)"

Role.java:

```
"package oblig3.moduls;
```

```
public class Role{
```

```
    private String roleFirstName;
```

```
    private String roleLastName;
```

```
    private Person actor;
```

```
    public Role(String roleFirstName, String roleLastName, Person actor){
```

```
        setRoleFirstName(roleFirstName);
```

```
        setRoleLastName(roleLastName);
```

```
        setActor(actor);
```

```
    }
```

```
    @Override public String toString() {
```

```
        return String.format("role: %s %s\nplayed by:\n%s",getRoleFirstName(), getRoleLastName(),  
getActor());}
```

```
    public String getRoleFirstName() {return roleFirstName;}
```

```
    public void setRoleFirstName(String roleFirstName) {this.roleFirstName = roleFirstName;}
```

```
    public String getRoleLastName() {return roleLastName;}
```

```
    public void setRoleLastName(String roleLastName) {this.roleLastName = roleLastName;}
```

```
public Person getActor() {return actor;}

public void setActor(Person actor) {this.actor = actor;}

}"
```

Movie.java:

```
"package oblig3.moduls;

import java.time.LocalDate;

public class Movie extends Production {

public Movie(String title, double runtime, String description, LocalDate releaseDate, Person director){

super(title, runtime,description,releaseDate,director);

}

@Override public String toString() {return super.toString();}

}"
```

Production.java:

```
"package oblig3.moduls;

import java.time.LocalDate;

import java.time.format.DateTimeFormatter;

import java.util.ArrayList;

public class Production{

private String title;

private double runtime;

private static final DateTimeFormatter correctFormat = DateTimeFormatter.ofPattern("dd.MM.yyyy");

private String description;

private LocalDate releaseDate;

private Person director;

private ArrayList<Role> roles = new ArrayList<>();

}
```

```
public Production (String title, double runtime,String description,LocalDate releaseDate,Person
director) {

this(title, description, releaseDate);

setRuntime(runtime);

setDirector(director);

}

public Production (String title) {setTitle(title);}
```

```
public Production(String title, String description, LocalDate releaseDate) {

this(title);

setDescription(description);

setReleaseDate(releaseDate);

}
```

```
public void addRoles(Role role){

roles.add(role);

};

public void addRoles(ArrayList<Role> listOfRoles) {roles.addAll(listOfRoles);};
```

```
@Override public String toString() {

return "title: "+getTitle()+"\ndescription: "+getDescription() +"\nrelease date:
"+getReleaseDate().format(correctFormat)

+"\nruntime in minutes: "+getRuntime()+"\ndirector: "+getDirector().getFullName()+"\n";

}
```

```
public String getFormattedReleaseDate() {if (releaseDate == null) {return "N/A";}return
releaseDate.format(correctFormat);}

public String getTitle() {return title;}

public void setTitle(String title) {this.title = title;}

public double getRuntime() {return runtime;}

public void setRuntime(double runtime) {if (runtime <= 0) {this.runtime = 0;} else {this.runtime =
runtime;}}
```

```

public LocalDate getReleaseDate() {return releaseDate;}

public void setReleaseDate(LocalDate releaseDate) {this.releaseDate = releaseDate;}

public String getDescription() {return description;}

public void setDescription(String description) {this.description = description;}

public Person getDirector() {return director;}

public void setDirector(Person director) {this.director = director;}

public ArrayList<Role> getRoles() {return new ArrayList<>{roles};}

}"

```

TVSeries.java:

```

"package oblig3.moduls;

```

```

import java.time.format.*;

```

```

import java.util.ArrayList;

```

```

import java.time.*;

```

```

public class TVSeries extends Production{

```

```

    private String title;

```

```

    private String description;

```

```

    private LocalDate releaseDate;

```

```

    private ArrayList<Episode> episodes= new ArrayList<>();

```

```

    private static final DateTimeFormatter correctFormat = DateTimeFormatter.ofPattern("dd.MM.yyyy");

```

```

    private double averageRunTime;

```

```

    private int numSeasons;

```

```

    private ArrayList<Role> cast = new ArrayList<>();

```

```

    public TVSeries(String title, String description, LocalDate releaseDate){

```

```

        super(title,description,releaseDate);

```

```

    }

```

```

    public ArrayList<Role> getCast(Class<?> clazz){

```

```
for (Role role:getRoles()){  
    if (role.getClass() == clazz){  
        cast.add(role);  
    }  
}  
return new ArrayList<>(cast);  
}
```

```
public void addEpisode(Episode episode){  
    if (episode == null) {  
        System.out.println("\nepisode does not exist");  
        return;  
    }  
}
```

```
if(episode.getSeasonNumber()>numSeasons+1){  
    System.out.printf("\nepisode %d season %d's season number is too high. episode not added.\n" +  
        "the episode in question:\n", episode.getEpisodeNumber(), episode.getSeasonNumber());  
    System.out.println(episode);  
    return;  
}
```

```
if(episode.getSeasonNumber()>numSeasons){  
    numSeasons = episode.getSeasonNumber();  
}
```

```
episodes.add(episode);  
updateAverageRunTime();  
}
```

```
private void updateAverageRunTime(){  
    double total = 0;
```

```

if(episodes.isEmpty()){
    averageRunTime = 0;
}else {
    for (Episode episode : episodes) {
        total += episode.getRuntime();
    }
}
averageRunTime = total/getEpisodes().size();
};

```

```

public ArrayList<Episode> getEpisodesInSeason(int season){
    ArrayList<Episode> showEpisodes = new ArrayList<>();
    for (Episode episode : episodes) {
        if(episode.getSeasonNumber() == season){
            showEpisodes.add(episode);
        }
    }
    return new ArrayList<>(showEpisodes);
}

```

```

@Override public String toString() {
    return "TV series title: "+getTitle()+"\ndescription: "+getDescription()+"\nrelease date: "
        +getReleaseDate().format(correctFormat)
        +"\nnumber of episodes: "+getEpisodes().size()+"\n"+getCast(TVSeries.class);
}

```

```

public ArrayList<Episode> getEpisodes() {return new ArrayList<>(episodes);}
public LocalDate getReleaseDate() {return releaseDate;};
public void setReleaseDate(LocalDate releaseDate) {this.releaseDate = releaseDate;}
public String getDescription() {return description;}
public void setDescription(String description) {this.description = description;}

```



```
public String getTitle() {return title;}

public void setTitle(String title) {this.title = title;}

public double getAverageRunTime() {return averageRunTime;}

public int getNumSeasons() {return numSeasons;}

}"
```

Person.java:

```
"package oblig3.moduls;
```

```
public class Person {

private String firstName;

private String lastName;

private int age;
```

```
public Person(String firstName, String lastName, int age){

setFirstName(firstName);

setLastName(lastName);

setAge(age);

}
```

```
@Override public String toString() {return String.format("full name: %s\nage: %d\n", getFullName(),
getAge());}
```

```
public String getFullName() {return String.format("%s %s",getFirstName(),getLastName());}

public void setFirstName(String firstName) {this.firstName = firstName;}

public void setLastName(String lastName) {this.lastName = lastName;}

public void setAge(int age) {this.age = age;}

public int getAge() {return age;}

public String getFirstName() {return firstName;}

public String getLastName() {return lastName;}

}"
```

Episode.java:

```
"package oblig3.moduls;
```

```
import java.time.LocalDate;
```

```
public class Episode extends Production{
```

```
private int episodeNumber;
```

```
private int seasonNumber;
```

```
public Episode(String episodeTitle, int episodeNumber, int seasonNumber, double runtime,
```

```
String description, LocalDate releaseDate, Person director){
```

```
super(episodeTitle, runtime,description,releaseDate,director);
```

```
setEpisodeNumber(episodeNumber);
```

```
setSeasonNumber(seasonNumber);
```

```
}
```

```
public Episode(String episodeTitle, int episodeNumber, int seasonNumber){
```

```
this(episodeTitle, episodeNumber, seasonNumber,0,null,null,null);
```

```
}
```

```
@Override public String toString() {
```

```
return "\n\ttitle: "+ getTitle()+"\n\tepisode number: "+getEpisodeNumber()+"\n\tseason number: "+getSeasonNumber()
```

```
+ "\n\truntime in minutes: "+getRuntime()+"\n";}
```

```
public int getEpisodeNumber() {return episodeNumber;}
```

```
public void setEpisodeNumber(int episodeNumber) {if (episodeNumber<0){this.episodeNumber = 0;
```

```
System.out.println("episode number cant be under 0. episode number set to 0.");}
```

```
else{this.episodeNumber = episodeNumber;}
```

```
}
```

```
public int getSeasonNumber() {return seasonNumber;}
```

```

public void setSeasonNumber(int seasonNumber) {if(seasonNumber<0){this.seasonNumber = 0;
System.out.println("season number cant be under 0. season number set to 0");}
else{this.seasonNumber = seasonNumber;}
}
}

```

Main.java:

```

"package oblig3;

import oblig3.moduls.*;

import java.time.*;

import java.util.ArrayList;

import java.util.Random;

public class Main {

public static void main(String[] args) {

LocalDate date = LocalDate.of(1965,11,8);

TVSeries noe = new TVSeries("Days of our Lives","noe", date);

Random random = new Random();

ArrayList<Person> personArrayList = new ArrayList<>();

Person ww = new Person("Walter", "White", 55);

Person jp = new Person("Jesse", "Pinkman", 26);

Person dd = new Person("Ding", "Dong",42);

Person rdjr = new Person("robert", "downey jr", 50);

Person jd = new Person("jân", "dâw", 67);

Person pdf = new Person("pita", "d. foya", 69);

personArrayList.add(ww);

personArrayList.add(jp);

personArrayList.add(dd);

personArrayList.add(jd);

personArrayList.add(pdf);

```

```
Movie honhon = new Movie("honhon",234,"wiwi",date,  
personArrayList.get(random.nextInt(0, personArrayList.size())));
```

```
Role ts = new Role("tony", "stark", rdjr);  
Role role1 = new Role("ada", "kadabra", ww);  
Role role2 = new Role("wewe", "wiwi", jp);  
Role role3 = new Role("kaka", "koko", dd);  
ArrayList<Role> listOfRoles = new ArrayList<>();  
listOfRoles.add(role1);  
listOfRoles.add(role2);  
listOfRoles.add(role3);  
listOfRoles.add(ts);
```

```
int episode = 1;  
int season = 1;  
int episodeNr = 1;  
int aarFoer = date.getYear();  
while(episodeNr<=14430){  
if (date.getYear()>aarFoer){  
aarFoer = date.getYear();  
episode=1;  
season++;}  
if (!(date.getDayOfWeek() == DayOfWeek.SATURDAY || date.getDayOfWeek() ==  
DayOfWeek.SUNDAY)) {  
Episode ep = new Episode("mao mao " + episode, episode, season, random.nextInt(20, 30),  
"bao bao",date,personArrayList.get(random.nextInt(0, personArrayList.size())));  
ep.addRoles(listOfRoles.get(random.nextInt(0, listOfRoles.size())));  
noe.addEpisode(ep);  
  
episode++;  
episodeNr++;
```

```

}
date = date.plusDays(1);
}
listOfRoles.remove(ts);
honhon.addRoles(ts);
honhon.addRoles(listOfRoles);
System.out.println(honhon);
for (Role show: honhon.getRoles()){
System.out.println(show);
}

System.out.println(noel);
ArrayList<Episode> season42 = noel.getEpisodesInSeason(42);
System.out.println("--- episodes in season 42 ---");
System.out.printf("%-15s %-10s %-10s %-10s %-15s %-15s %-15s %s\n",
"Title", "Episode", "Season", "Runtime", "Description", "Release Date", "Director", "Role");

for (Episode ep : season42) {
System.out.printf("%-15s %-10d %-10d %-3.0f %-6s %-15s %-15s %s\n",
ep.getTitle(), ep.getEpisodeNumber(), ep.getSeasonNumber(),
ep.getRuntime(), "min", ep.getDescription(), ep.getFormattedReleaseDate(), ep.getDirector().getFullNa
me());
}
System.out.println("total episodes in this season: " + season42.size()+"\n");
}
}"

```

oppgave 2.5 og 2.6 er ferdig. men på 2.7 sliter jeg med

```

"public ArrayList<Role> getCast(Class<?> clazz){
for (Role role: getRoles()){
if (role.getClass() == clazz){

```

```

cast.add(role);
}
}

return new ArrayList<>(cast);
}" og "@Override public String toString() {
return "TV series title: "+getTitle()+"\ndescription: "+getDescription()+"\nrelease date: "
+getReleaseDate().format(correctFormat)
+"\nnumber of episodes: "+getEpisodes().size()+"\n"+getCast(TVSeries.class);
}" ved å sjekke om instansene i getRoles tilhører klassen TVSeries og returnere en kopi av en ny liste
med tilhørende roller. hjelp takk

```

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Hei! Det er kjempebra at du har kommet så langt. Du har god kontroll på klasseoppsett og nedarving.

Når det gjelder Oppgave 2.7, ser det ut til at du har støtt på en logisk utfordring knyttet til hvor rollene faktisk befinner seg i systemet ditt. La oss bryte ned problemet litt:

Refleksjon rundt getCast()

I oppgaveteksten står det:

"Du må altså hente alle roller fra alle episoder som tilhører TV-serien."

I koden din for TVSeries.java forsøker du nå å bruke getRoles() og sjekke om klassen er TVSeries.class. Her er det et par ting vi bør se på:

1. **Hvor ligger dataene?** En TVSeries har en liste med Episode-objekter. Hver Episode er en Production, og siden Production har listen roles, betyr det at det er hver enkelt **episode** som sitter på informasjonen om hvilke roller som var med i akkurat den episoden.
2. **Logikken i getCast():** Akkurat nå henter getCast() roller fra TVSeries sin egen roles-liste (arvet fra Production). Men inneholder denne listen egentlig noe? I Main.java ser jeg at du legger til roller i hver Episode, ikke direkte i TVSeries-objektet.

Spørsmål til videre problemløsning:

- Siden målet er å samle alle roller fra hele serien, hvordan kan du bruke listen din med episodes for å få tak i rollene som ligger gjemt inni hver enkelt episode?
- Hvilken kontrollstruktur (løkke) trenger du for å gå igjennom alle episodene i serien?
- Når du er inne i en episode, hvilken metode har du allerede tilgjengelig (arvet fra Production) for å hente ut rollene i den episoden?

Et lite tips om toString():

I din toString() kaller du getCast(TVSeries.class). Siden getCast() er ment å returnere en liste over alle skuespillere i hele serien, trenger du egentlig å sende inn TVSeries.class som et argument der? Oppgaven ber om alle roller fra alle episoder, uavhengig av klasse-type.

Et lite teknisk obs: I TVSeries har du deklartert variabler som title, description og releaseDate på nytt. Siden TVSeries extends Production, og Production allerede har disse feltetene, bør du vurdere om det er nødvendig å ha dem i TVSeries også, eller om du kan stole på de du arver. Dette kan spare deg for forvirring når du bruker getTitle() og lignende!

Hva tenker du er den beste måten å "samle sammen" listene fra alle episodene til én stor liste?

Du sa

kan jeg få et generell eksempel relatert til problemet?

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Absolutt! Siden problemet ditt handler om å hente data som ligger "nøstet" inne i en liste av objekter, kan vi se på et helt annet eksempel: et **Bibliotek** som har mange **Bokhyller**, der hver bokhylle inneholder flere **Bøker**.

Hvis vi ønsker en liste over *alle* bøkene i hele biblioteket, må vi gå igjennom hver hylle og samle sammen bøkene.

Eksempel: Hente alle elementer fra undersider

Se for deg at vi har en klasse Shelf som har en liste med titler, og en klasse Library som holder på alle hyllene.

Java

// En hylle som inneholder en liste med boktitler

```
class Shelf {  
  
    private ArrayList<String> bookTitles = new ArrayList<>();  
  
    public void addBook(String title) {  
        bookTitles.add(title);  
    }  
  
    // Returnerer titlene på denne spesifikke hyllen  
    public ArrayList<String> getBookTitles() {  
        return new ArrayList<>(bookTitles);  
    }  
}
```

```
}  
}
```

// Biblioteket som inneholder mange hyller

```
class Library {
```

```
    private ArrayList<Shelf> allShelves = new ArrayList<>();
```

// Mål: Hente ALLE titler fra ALLE hyller i hele biblioteket

```
    public ArrayList<String> getAllTitlesInLibrary() {
```

```
        ArrayList<String> completeList = new ArrayList<>();
```

// 1. Gå igjennom hver hylle i biblioteket

```
    for (Shelf shelf : allShelves) {
```

// 2. Hent ut listen med titler fra den aktuelle hyllen

```
        ArrayList<String> titlesFromThisShelf = shelf.getBookTitles();
```

// 3. Legg alle disse titlene inn i den store samlelisten

```
        completeList.addAll(titlesFromThisShelf);
```

```
    }
```

```
    return completeList;
```

```
}
```

```
}
```

Hvorfor er dette relevant for din getCast()?

I din oppgave tilsvarer:

- Library din TVSeries.
- Shelf din Episode.
- bookTitles dine roles (som ligger inne i hver episode).

Noen ting å legge merke til i eksempelet:

1. **addAll()-metoden:** Dette er en veldig nyttig metode i ArrayList. I stedet for å legge til ett og ett element med add(), kan du legge til en hel liste i en annen liste med én kommando.
2. **Løkken:** Vi må "besøke" hvert underobjekt (episoden) for å få tak i dataene som bor der.
3. **Lokal liste:** Vi lager en midlertidig liste (completeList) inni metoden som vi fyller opp og returnerer til slutt.

Ser du hvordan du kan overføre denne logikken til TVSeries for å gå igjennom episodes-listen din i stedet for å lete i TVSeries sin egen roles-liste?

Du sa

kan du legge til kode som unngår duplikasjon

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

For å unngå duplikater i en liste i Java, er den vanligste og mest effektive måten å bruke et **Set**.

I programmering er et Set (mengde) en samling som per definisjon ikke kan inneholde duplikater. Hvis du prøver å legge til noe som allerede finnes der, vil den rett og slett ignorere det.

Her er hvordan logikken fra bibliotek-eksempelet kan utvides for å sikre at hver tittel bare dukker opp én gang:

Eksempel: Unngå duplikater med HashSet

Java

```
import java.util.ArrayList;
```

```
import java.util.HashSet;
```

```
class Library {  
    private ArrayList<Shelf> allShelves = new ArrayList<>();  
  
    public ArrayList<String> getUniqueTitles() {  
        // 1. Lag et HashSet for å holde på unike elementer  
        HashSet<String> uniqueTitlesSet = new HashSet<>();  
  
        for (Shelf shelf : allShelves) {  
            // 2. Legg til alle elementene fra hyllen inn i settet  
            // HashSet vil automatisk kaste bort duplikater  
            uniqueTitlesSet.addAll(shelf.getBookTitles());  
        }  
    }  
}
```

```

    }

    // 3. Konverter settet tilbake til en ArrayList (siden metoden krever det)
    return new ArrayList<>(uniqueTitlesSet);
}
}

```

Hvorfor fungerer dette?

- **HashSet:** Når du bruker `addAll()` på et `HashSet`, sjekker Java om objektet allerede eksisterer i samlingen. For at dette skal fungere optimalt med egne klasser (som din `Role`-klasse), er det viktig at Java vet hva som gjør to objekter "like".
- **Likhet (`equals/hashCode`):** I eksempelet over brukes `String`. Java vet allerede hvordan den sammenligner to strenger. Hvis du skal bruke dette på `Role`, vil Java som standard sjekke om det er det *nøyaktig samme objektet* (samme plass i minnet).

Hvordan overføre dette til din oppgave?

I din `getCast()`-metode i `TVSeries`:

1. Opprett et `HashSet<Role>`.
2. Loop gjennom episodene dine.
3. Hent rollene fra hver episode og legg dem til i settet ditt.
4. Returner en ny `ArrayList` basert på dette settet.

Et lite spørsmål til refleksjon: Oppgaven din nevner faktisk: *"Du vil mest sannsynlig få samme skuespiller og rolle flere ganger... dette er OK"*.

Hvis du likevel ønsker å fjerne duplikater for din egen lærings del: Tror du det er best å returnere en liste der den samme karakteren (f.eks. "Tony Stark") kun vises én gang totalt, eller er det viktig å se hvilke episoder de faktisk var med i?

Vil du at jeg skal forklare mer om hvordan `HashSet` fungerer sammenlignet med `ArrayList`?

Du sa

ja

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Det er et kjempefint tema å dykke ned i, for valget mellom `ArrayList` og `HashSet` er noe av det mest grunnleggende (og nyttigste) man gjør som Java-utvikler.

Her er de viktigste forskjellene forklart enkelt:

1. ArrayList (En ordnet liste)

Tenk på ArrayList som en vanlig kø eller en handleliste.

- **Rekkefølge:** Den tar vare på rekkefølgen elementene ble lagt inn i. Det første elementet er alltid på plass 0.
- **Duplikater:** Den bryr seg ikke om du har det samme elementet ti ganger. Hvis du legger til "Tony Stark" ti ganger, har du ti "Tony Stark"-objekter i listen.
- **Søk:** Hvis du skal finne ut om et element finnes i en ArrayList med 1 million elementer, må Java i verste fall sjekke hvert eneste ett fra start til slutt. Dette tar tid ().

2. HashSet (En unik mengde)

Tenk på HashSet som en stor sekk med unike klinkekuler.

- **Rekkefølge:** Den bryr seg **ikke** om rekkefølgen. Når du legger noe i et HashSet, kan det havne "hvor som helst".
- **Duplikater:** Den tillater kun én av hver. Hvis du prøver å legge til "Tony Stark" ti ganger, vil settet fortsatt bare inneholde én instans.
- **Søk:** Her er HashSet lynraskt. Ved hjelp av en matematisk funksjon (hashing) vet Java nesten nøyaktig hvor i minnet objektet skal ligge. Det betyr at uansett om du har 10 eller 1 million elementer, tar det like kort tid å sjekke om noe finnes der ().

Oppsummeringstabell

Egenskap	ArrayList	HashSet
Tillater duplikater?	Ja	Nei
Bevarer rekkefølge?	Ja	Nei
Hastighet (Søk/Oppslag)	Treg (må gå gjennom alt)	Ekstremt rask
Brukstilfelle	Når rekkefølge er viktig	Når du vil unngå kopier

Hvordan velger du i din oppgave?

Siden oppgaveteksten for 2.7 sier:

"Du vil mest sannsynlig få samme skuespiller og rolle flere ganger [...] dette er OK"

...så er det mest korrekt i henhold til spesifikasjonen å bruke en ArrayList. Men hvis du vil imponere eller lage et mer oversiktlig resultat for brukeren, er HashSet veien å gå for å fjerne duplikater.

Vil du se hvordan du kan endre koden i `getCast()` slik at den looper gjennom episodene og samler alle rollene i en av disse listetypene?

Du sa

```
private ArrayList<Role> cast;
```

```

public ArrayList<Role> getCast(){
    HashSet<Role> uniqueRoles = new HashSet<>();
    for(Episode episode:getEpisodes()){
        uniqueRoles.addAll(episode.getRoles());
    }
    cast = new ArrayList<>(uniqueRoles);
    return cast;
}

```

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Dette ser veldig bra ut! Du har nå forstått logikken med å gå igjennom hver Episode og samle sammen rollene i et HashSet for å unngå duplikater. Dette er en veldig elegant løsning på problemet.

Her er et par små detaljer du bør vurdere for å gjøre koden din helt "proff":

1. Lokal variabel vs. Instansvariabel

I koden din lagrer du resultatet i `private ArrayList<Role> cast;`. Siden en TV-serie kan få nye episoder (og dermed nye roller) over tid, er det ofte bedre å ikke lagre denne listen som en fast variabel i klassen, men heller generere den "der og da" når metoden kalles.

Hvis du beholder den som en instansvariabel, risikerer du at den blir utdatert hvis du legger til en episode etter at du har kalt `getCast()` én gang. Ved å fjerne `private ArrayList<Role> cast;` fra toppen av klassen og bare returnere listen direkte, sørger du for at dataene alltid er ferske.

2. Bruk av `getRoles()`

Siden `Episode` arver fra `Production`, og du har `getRoles()`-metoden i `Production`, fungerer `episode.getRoles()` helt utmerket. Husk at din `getRoles()` i `Production` allerede returnerer en kopi (`return new ArrayList<>(roles);`), noe som er god praksis for å beskytte dataene.

3. Oppgave 2.7 spesifikasjon

Oppgaven ber deg om å:

"Skriv så ut navnet på rollene, og skuespillerne som spiller disse."

Siden du nå returnerer en liste med `Role`-objekter, har du alt du trenger. I `Main.java` (eller der du tester), kan du nå enkelt løpe igjennom denne listen og skrive ut informasjonen.

Slik kan den ferdige metoden se ut uten den unødvendige instansvariabelen:

Java

```

public ArrayList<Role> getCast() {
    HashSet<Role> uniqueRoles = new HashSet<>();

```

```

    for (Episode episode : episodes) {
        uniqueRoles.addAll(episode.getRoles());
    }

    return new ArrayList<>(uniqueRoles);
}

```

Et lite kontrollspørsmål før du går videre:

Når du nå skriver ut denne listen i Main, bruker du toString()-metoden du laget i Oppgave 2.6? Og er du fornøyd med hvordan Role sin toString() presenterer både karakterens navn og skuespillerens navn?

Vil du at vi skal se på hvordan du kan formatere utskriften i Main for å få det til å se ut som en fin oversikt?

Du sa

```
package oblig3.moduls;
```

```

import java.time.format.*;
import java.util.ArrayList;
import java.time.*;
import java.util.HashSet;

```

```

public class TVSeries extends Production{
    private String title;
    private String description;
    private LocalDate releaseDate;
    private ArrayList<Episode> episodes= new ArrayList<>();
    private static final DateTimeFormatter correctFormat = DateTimeFormatter.ofPattern("dd.MM.yyyy");
    private double averageRunTime;
    private int numSeasons;

    public TVSeries(String title, String description, LocalDate releaseDate){
        super(title,description,releaseDate);
    }
}

```

```
}
```

```
public ArrayList<Role> getCast(){  
    HashSet<Role> uniqueRoles = new HashSet<>();  
    for(Episode episode:getEpisodes()){  
        uniqueRoles.addAll(episode.getRoles());  
    }  
    return new ArrayList<>(uniqueRoles);  
}
```

```
public void addEpisode(Episode episode){  
    if (episode == null) {  
        System.out.println("\nepisode does not exist");  
        return;  
    }
```

```
    if(episode.getSeasonNumber()>numSeasons+1){  
        System.out.printf("\nepisode %d season %d's season number is too high. episode not added.\n" +  
            "the episode in question:\n", episode.getEpisodeNumber(), episode.getSeasonNumber());  
        System.out.println(episode);  
        return;  
    }
```

```
    if(episode.getSeasonNumber()>numSeasons){  
        numSeasons = episode.getSeasonNumber();  
    }
```

```
    episodes.add(episode);  
    updateAverageRunTime();  
}
```

```

private void updateAverageRunTime(){
double total = 0;
if(epsisodes.isEmpty()){
averageRunTime = 0;
}else {
for (Episode episode : epsisodes) {
total += episode.getRuntime();
}
}
averageRunTime = total/getEpisodes().size();
};

```

```

public ArrayList<Episode> getEpisodesInSeason(int season){
ArrayList<Episode> showEpisodes = new ArrayList<>();
for (Episode episode : epsisodes) {
if(episode.getSeasonNumber() == season){
showEpisodes.add(episode);
}
}
return new ArrayList<>(showEpisodes);
}

```

```

@Override public String toString() {
return "TV series title: "+getTitle()+"\ndescription: "+getDescription()+"\nrelease date: "
+getReleaseDate().format(correctFormat)
+"\nnumber of episodes: "+getEpisodes().size()+"\n";
}

```

```

public ArrayList<Episode> getEpisodes() {return new ArrayList<>(epsisodes);}
public LocalDate getReleaseDate() {return releaseDate;};
public void setReleaseDate(LocalDate releaseDate) {this.releaseDate = releaseDate;}

```

```

public String getDescription() {return description;}

public void setDescription(String description) {this.description = description;}

public String getTitle() {return title;}

public void setTitle(String title) {this.title = title;}

public double getAverageRunTime() {return averageRunTime;}

public int getNumSeasons() {return numSeasons;}

}

```

```

System.out.printf("the cast playing in %s:\n%s",noe.getTitle(),noe.getCast());

```

nå da?

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Dette ser veldig solid ut! Nå har du en getCast()-metode som følger logikken i oppgaven ved å hente roller fra alle episodene, og du bruker HashSet for å sikre at lista er ryddig og uten duplikater.

Når du nå kjører System.out.printf, er det imidlertid én ting du vil legge merke til i utskriften: **Formateringen.**

Hva skjer ved utskrift?

Når du printer noe.getCast(), som er en ArrayList, vil Java kalle toString() på selve listen. Dette resulterer i en utskrift som ser omtrent slik ut: [role: Tony Stark played by: Robert Downey Jr, role: Walter White played by: Bryan Cranston, ...]

Det fungerer teknisk sett, men det kan bli litt vanskelig å lese hvis listen er lang.

Forslag til en ryddigere presentasjon

Siden du i Main ønsker å vise frem "casten", kan du vurdere å gå igjennom listen med en løkke for å printe hver rolle på en ny linje. Dette gjør det mye mer oversiktlig for brukeren.

Her er et generelt eksempel på hvordan man kan iterere over en slik liste for utskrift:

Java

```

System.out.println("Cast members in " + myProduction.getTitle() + ":");

```

```

// Gå igjennom listen vi får fra metoden

```

```

for (Role role : myProduction.getCast()) {

```

```

    // Her brukes Role-klassens egen toString() automatisk

```



```
        System.out.println("- " + role);
    }
}
```

En siste sjekk av TVSeries

Jeg la merke til en liten detalj i TVSeries-klassen din som kan skape krøll senere:

Du har feltet `private String title;` (og `description`, `releaseDate`) i `TVSeries`, men disse finnes også i `Production`. Når du kaller `getTitle()` i `toString()`, kan det hende den henter verdien fra feil variabel hvis de ikke er synkronisert.

Siden `TVSeries` extends `Production`, har du allerede tilgang til disse variablene via `super`-klassen. Du kan trygt fjerne disse deklarasjonene fra `TVSeries` og bare bruke `super(...)` i konstruktøren slik du allerede gjør. Da unngår du såkalt "variable shadowing".

Er du fornøyd med hvordan utskriften ser ut i konsollen nå, eller vil du ha hjelp til å formatere den mer som en tabell slik du gjorde med episodene?

Du sa

```
package oblig3.moduls;
```

```
import java.time.format.*;
```

```
import java.util.ArrayList;
```

```
import java.time.*;
```

```
import java.util.HashSet;
```

```
public class TVSeries extends Production{
```

```
    private LocalDate releaseDate;
```

```
    private ArrayList<Episode> episodes= new ArrayList<>();
```

```
    private static final DateTimeFormatter correctFormat = DateTimeFormatter.ofPattern("dd.MM.yyyy");
```

```
    private double averageRunTime;
```

```
    private int numSeasons;
```

```
    public TVSeries(String title, String description, LocalDate releaseDate){
```

```
        super(title,description,releaseDate);
```

```
    }
```

```
    public ArrayList<Role> getCast(){
```

```

HashSet<Role> uniqueRoles = new HashSet<>();
for(Episode episode:getEpisodes()){
    uniqueRoles.addAll(episode.getRoles());
}
return new ArrayList<>(uniqueRoles);
}

```

```

public void addEpisode(Episode episode){
    if (episode == null) {
        System.out.println("\nepisode does not exist");
        return;
    }
}

```

```

if(episode.getSeasonNumber()>getNumSeasons()+1){
    System.out.printf("\nepisode %d season %d's season number is too high. episode not added.\n" +
        "the episode in question:\n", episode.getEpisodeNumber(), episode.getSeasonNumber());
    System.out.println(episode);
    return;
}

```

```

if(episode.getSeasonNumber()>getNumSeasons()){
    numSeasons = episode.getSeasonNumber();
}

```

```

episodes.add(episode);
updateAverageRunTime();
}

```

```

private void updateAverageRunTime(){
    double total = 0;
    if(episodes.isEmpty()){

```

```

averageRunTime = 0;
}else {
for (Episode episode : episodes) {
total += episode.getRuntime();
}
}
averageRunTime = total/getEpisodes().size();
};

```

```

public ArrayList<Episode> getEpisodesInSeason(int season){
ArrayList<Episode> showEpisodes = new ArrayList<>();
for (Episode episode : episodes) {
if(episode.getSeasonNumber() == season){
showEpisodes.add(episode);
}
}
return new ArrayList<>(showEpisodes);
}

```

```

@Override public String toString() {
return "TV series title: "+getTitle()+"\ndescription: "+getDescription()+"\nrelease date: "
+getReleaseDate().format(correctFormat)
+"\nnumber of episodes: "+getEpisodes().size()+"\n";
}

```

```

public ArrayList<Episode> getEpisodes() {return new ArrayList<>(episodes);}
public LocalDate getReleaseDate() {return releaseDate;};
public void setReleaseDate(LocalDate releaseDate) {this.releaseDate = releaseDate;}
public double getAverageRunTime() {return averageRunTime;}
public int getNumSeasons() {return numSeasons;}
}

```

bedre?

Programmering 2 Chat-Assistent

Egendefinert Gem

Programmering 2 Chat-Assistent said

Dette ser mye bedre og ryddigere ut! Ved å fjerne de overflødige variablene (title og description) utnytter du nå arven fra Production korrekt. Dette kalles **innkapsling** og gjenbruk av kode, som er helt sentralt i objektorientert programmering.

Her er en kjapp vurdering av de siste endringene dine:

Hva som er bra:

- **Arv:** Du bruker nå getTitle() og getDescription() som arves fra Production. Dette sikrer at du alltid henter data fra "kilden".
- **Logikk i addEpisode:** Du har byttet ut numSeasons med getNumSeasons() i sjekken din, som er god praksis.
- **getCast():** Metoden er nå effektiv, fjerner duplikater ved hjelp av HashSet, og henter data korrekt fra alle episoder.

Et lite tips for 100% nøyaktighet:

Jeg ser at du fortsatt har private LocalDate releaseDate; stående i TVSeries.java. Siden Production allerede har en releaseDate (og du sender denne til super i konstruktøren), kan du også fjerne denne linjen samt getReleaseDate- og setReleaseDate-metodene i TVSeries.

Hvis du beholder den i TVSeries, vil TVSeries ha sin egen kopi av datoen som kan være ulik den som ligger i Production. Dette kalles "variable shadowing" og kan føre til at toString() viser en dato mens en annen metode viser noe annet.

Oppsummering av dataflyten din:

Når du kaller getCast(), skjer dette:

1. En tom "beholder" (HashSet) lages.
2. Du besøker hver Episode.
3. Hver Episode gir fra seg sin liste med Role-objekter.
4. HashSet beholder kun de unike rollene.
5. Du sender ut resultatet som en ren liste.

Veien videre:

Nå som klassestrukturen din er på plass og getCast() fungerer, er du i mål med de tekniske kravene i oppgaven!

Vil du at jeg skal hjelpe deg med å se på hvordan du kan formatere utskriften av cast-listen i Main slik at den ser proff ut i konsollen, eller er du klar for å levere?